



机器学习实验二：支持向量机模型

姓名：冯怡然

学号：2411434

班级：人工智能特色班

日期：2026 年 4 月 29 日

南开大学 · 机器学习实验

1 实验目的

1. 理解线性支持向量机的几何意义与优化目标。
2. 掌握对偶问题建模与参数 w, b, α 的求解关系。
3. 在二维数据上观察线性可分与线性不可分场景下的分类边界差异。
4. 比较不同训练方法与超参数 (β /步长、 C) 对速度与分类效果的影响。

2 实验内容

1. 基础任务：生成两类二维高斯数据，每类 100 个样本，完成 SVM 训练并可视化。
2. 场景 A（线性可分）：第二类中心设为 (6,6)。
3. 场景 B（线性不可分）：第二类中心设为 (3,3)。
4. 附加题 1：软间隔问题的数学推导
5. 附加题 2：学习增广拉格朗日法（ALM），调节 β （程序中记作 `rho`）与步长 `step`，观察速度变化。
6. 附加题 3：调节惩罚系数 C ，观察分类结果变化。

3 实验原理

3.1 线性 SVM 模型

分类函数：

$$f(x) = \text{sign}(w^\top x + b)$$

软间隔原始问题：

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i, \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

3.2 对偶形式

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j x_i^\top x_j \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^m \alpha_i y_i = 0 \end{aligned}$$

并有

$$w = \sum_{i=1}^m \alpha_i y_i x_i$$

3.3 训练方法

本实验代码支持两种训练器：

- SMO（序列最小优化）；
- ALM（增广拉格朗日法，带投影与步长控制）。

4 SVM 基础训练

4.1 代码实现

4.1.1 参数设置和训练方法选择

```
1 clc; clear; close all; rng(1);
2 % 训练方法二选一：'smo' 或 'alm'
3 method = 'smo';
4 % 可调参数
5 params = struct();
6 params.C = 1;           % 软间隔系数
7 params.tol = 1e-4;       % 收敛阈值
8 % SMO参数
9 params.maxPasses = 20;
10 % ALM参数
11 params.rho = 5;         % 增广项系数
12 params.lambda0 = 0;     % 拉格朗日乘子初值
13 params.outerMax = 80;   % 外层迭代次数
14 params.innerMax = 300;  % 内层迭代次数
15 params.step = 0.003;    % 梯度步长
```

参数和训练方法

4.1.2 线性可分分类运行

```
1 n = 100;                % 样本量大小
2 center1 = [1,1];        % 第一类数据中心
3 center2 = [6,6];        % 第二类数据中心
4 X = zeros(2*n,2);       % 2n *
5                           % 2的数据矩阵，每一行表示一个数据点，第一列表示x轴坐标，第二列表示y轴坐标
6 Y = zeros(2*n,1);       % 类别标签
7 X(1:n,:) = ones(n,1)*center1 + randn(n,2);
8 X(n+1:2*n,:) = ones(n,1)*center2 + randn(n,2);
9                           % 矩阵X的前n行表示类别1中数据，后n行表示类别2中数据
10 Y(1:n) = 1;
11 Y(n+1:2*n) = -1;        % 第一类数据标签为1，第二类为-1
12 X_sep = X; Y_sep = Y;   % 供附加题实验复用
13 figure(1)
14 set(gcf, 'Position', [1,1,700,600], 'color', 'w')
15 set(gca, 'FontSize', 18)
```

```

14 plot(X(1:n,1),X(1:n,2),'go','LineWidth',1,'MarkerSize',10);
15 hold on;
16 plot(X(n+1:2*n,1),X(n+1:2*n,2),'b*','LineWidth',1,'MarkerSize',10);
17 hold on;
18 xlabel('x axis');
19 ylabel('y axis');
20 legend('class 1','class 2');
21 title('Linear Separable Data');
22 ax1 = axis;
23 [w,b,alpha,C] =train_svm_choose(X,Y,method,params);
24 x1 = -2 : 0.00001 : 7;
25 y1 = ( -b * ones(1,length(x1)) - w(1) * x1 )/w(2);
26 y2 = ( ones(1,length(x1)) - b * ones(1,length(x1)) - w(1) * x1 )/w(2);
27 y3 = ( -ones(1,length(x1)) - b * ones(1,length(x1)) - w(1) * x1 )/w(2);
28 figure(2)
29 set(gcf,'Position',[1,1,700,600], 'color','w')
30 set(gca,'FontSize',18)
31 plot(X(1:n,1),X(1:n,2),'go','LineWidth',1,'MarkerSize',10); hold on;
32 plot(X(n+1:2*n,1),X(n+1:2*n,2),'b*','LineWidth',1,'MarkerSize',10); hold on;
33 plot(x1,y1,'k','LineWidth',1,'MarkerSize',10);hold on;
34 plot(x1,y2,'k-','LineWidth',1,'MarkerSize',10);hold on;
35 plot(x1,y3,'k-','LineWidth',1,'MarkerSize',10);hold on;
36 plot(X(alpha>0,1),X(alpha>0,2),'rs','LineWidth',1,'MarkerSize',10); hold on;
37 plot(X(alpha<C&alpha>0,1),X(alpha<C&alpha>0,2),'rs','MarkerFaceColor',
38 'r','LineWidth',1,'MarkerSize',10); hold on;
39 xlabel('x axis');
40 ylabel('y axis');
41 set(gca,'FontSize',10)
42 legend('class 1','class 2','classification surface','boundary','boundary','support
    vectors','support vectors on boundary');
43 title(['Linear Separable SVM - ', upper(method)]);
44 axis(ax1);

```

线性可分分类运行

4.1.3 线性不可分分类运行

```

1 n = 100; % 样本量大小
2 center1 = [1,1]; % 第一类数据中心
3 center2 = [3,3]; % 第二类数据中心
4 X = zeros(2*n,2); % 2n *
    2的数据矩阵，每一行表示一个数据点，第一列表示x轴坐标，第二列表示y轴坐标
5 Y = zeros(2*n,1); % 类别标签
6 X(1:n,:) = ones(n,1)*center1 + randn(n,2);
7 X(n+1:2*n,:) = ones(n,1)*center2 + randn(n,2);
    %矩阵X的前n行表示类别1中数据，后n行表示类别2中数据
8 Y(1:n) = 1;
9 Y(n+1:2*n) = -1; % 第一类数据标签为1，第二类为-1
10 X_nonsep = X; Y_nonsep = Y; % 供附加题实验复用
11 figure(3)

```

```

12 set(gcf,'Position',[1,1,700,600], 'color','w')
13 set(gca,'FontSize',18)
14 plot(X(1:n,1),X(1:n,2),'go','LineWidth',1,'MarkerSize',10); hold on;
15 plot(X(n+1:2*n,1),X(n+1:2*n,2),'b*','LineWidth',1,'MarkerSize',10); hold on;
16 xlabel('x axis');
17 ylabel('y axis');
18 legend('class 1','class 2');
19 title('Linear Inseparable Data');
20 ax3 = axis;
21 [w,b,alpha,C] = train_svm_choose(X,Y,method,params);
22 x1 = -2 : 0.00001 : 7;
23 y1 = ( -b * ones(1,length(x1)) - w(1) * x1 )/w(2);
24 y2 = ( ones(1,length(x1)) - b * ones(1,length(x1)) - w(1) * x1 )/w(2);
25 y3 = ( -ones(1,length(x1)) - b * ones(1,length(x1)) - w(1) * x1 )/w(2);
26 figure(4)
27 set(gcf,'Position',[1,1,700,600], 'color','w')
28 set(gca,'FontSize',18)
29 plot(X(1:n,1),X(1:n,2),'go','LineWidth',1,'MarkerSize',10); hold on;
30 plot(X(n+1:2*n,1),X(n+1:2*n,2),'b*','LineWidth',1,'MarkerSize',10); hold on;
31 plot(x1,y1,'k','LineWidth',1,'MarkerSize',10); hold on;
32 plot(x1,y2,'k-','LineWidth',1,'MarkerSize',10); hold on;
33 plot(x1,y3,'k-','LineWidth',1,'MarkerSize',10); hold on;
34 plot(X(alpha>0,1),X(alpha>0,2),'rs','LineWidth',1,'MarkerSize',10); hold on;
35 plot(X(alpha<C&alpha>0,1),X(alpha<C&alpha>0,2),
36 'rs','MarkerFaceColor','r','LineWidth',1,'MarkerSize',10); hold on;
37 xlabel('x axis');
38 ylabel('y axis');
39 set(gca,'FontSize',10)
40 legend('class 1','class 2','classification surface','boundary','boundary','support
    vectors','support vectors on boundary');
41 title(['Linear Inseparable SVM - ', upper(method)]);
42 axis(ax3);

```

线性不可分分类运行

4.1.4 训练方法调用

```

1 function [w,b,alpha,C] = train_svm_choose(X,Y,method,params)
2 C = params.C;
3 switch lower(method)
4     case 'smo'
5         [w,b,alpha] = train_svm_smo(X,Y,params);
6     case 'alm'
7         [w,b,alpha] = train_svm_alm(X,Y,params);
8     otherwise
9         error('Unknown method. Use ''smo'' or ''alm''.');
10 end
11 end

```

训练方法调用

4.1.5 序列最小优化 (SMO) 方法实现

```
1 function [w,b,alpha] = train_svm_smo(X,Y,params)
2 C = params.C;
3 tol = params.tol;
4 maxPasses = params.maxPasses;
5 m = size(X,1);
6 K = X*X';
7 alpha = zeros(m,1);
8 b = 0;
9 passes = 0;
10 while passes < maxPasses
11     num_changed_alphas = 0;
12     for i = 1:m
13         f_i = sum(alpha .* Y .* K(:,i)) + b;
14         E_i = f_i - Y(i);
15         if ((Y(i)*E_i < -tol && alpha(i) < C) || (Y(i)*E_i > tol && alpha(i) > 0))
16             j = randi(m);
17             while j == i
18                 j = randi(m);
19             end
20             f_j = sum(alpha .* Y .* K(:,j)) + b;
21             E_j = f_j - Y(j);
22             alpha_i_old = alpha(i);
23             alpha_j_old = alpha(j);
24             if Y(i) ~= Y(j)
25                 L = max(0, alpha(j) - alpha(i));
26                 H = min(C, C + alpha(j) - alpha(i));
27             else
28                 L = max(0, alpha(i) + alpha(j) - C);
29                 H = min(C, alpha(i) + alpha(j));
30             end
31             if L == H
32                 continue;
33             end
34             eta = 2*K(i,j) - K(i,i) - K(j,j);
35             if eta >= 0
36                 continue;
37             end
38             alpha(j) = alpha(j) - Y(j)*(E_i - E_j)/eta;
39             alpha(j) = min(H, max(L, alpha(j)));
40             if abs(alpha(j) - alpha_j_old) < 1e-6
41                 alpha(j) = alpha_j_old;
42                 continue;
43             end
44             alpha(i) = alpha(i) + Y(i)*Y(j)*(alpha_j_old - alpha(j));
45             b1 = b - E_i ...
46                 - Y(i)*(alpha(i)-alpha_i_old)*K(i,i) ...
47                 - Y(j)*(alpha(j)-alpha_j_old)*K(i,j);
48             b2 = b - E_j ...
```

```

49         - Y(i)*(alpha(i)-alpha_i_old)*K(i,j) ...
50         - Y(j)*(alpha(j)-alpha_j_old)*K(j,j);
51     if alpha(i) > 0 && alpha(i) < C
52         b = b1;
53     elseif alpha(j) > 0 && alpha(j) < C
54         b = b2;
55     else
56         b = (b1 + b2)/2;
57     end
58     num_changed_alphas = num_changed_alphas + 1;
59 end
60 end
61 if num_changed_alphas == 0
62     passes = passes + 1;
63 else
64     passes = 0;
65 end
66 end
67 w = X' * (alpha .* Y);
68 end

```

SMO 实现

4.1.6 拉格朗日增广 (ALM) 方法实现

```

1 function [w,b,alpha] = train_svm_alm(X,Y,params)
2 % 对偶:
3 % min 0.5*alpha'*Q*alpha - 1'*alpha
4 % s.t. Y'*alpha = 0, 0<=alpha<=C
5 C = params.C;
6 tol = params.tol;
7 rho = params.rho;
8 lambda = params.lambda0;
9 outerMax = params.outerMax;
10 innerMax = params.innerMax;
11 step0 = params.step;
12 m = size(X,1);
13 Q = (Y*Y').*(X*X');
14 alpha = zeros(m,1);
15 for out = 1:outerMax
16     for in = 1:innerMax
17         eq = Y' * alpha;
18         grad = Q*alpha - ones(m,1) + (lambda + rho*eq)*Y;
19         if in == 1 step = step0; end % 自适应步长 (Barzilai-Borwein + 回退)
20         a_old = alpha; g_old = grad;
21         % 投影梯度一步
22         alpha_try = min(C, max(0, alpha - step*grad));
23         % 简单回退线搜索: 若目标没降就缩步长
24         f_old = 0.5*(alpha'*(Q*alpha)) - sum(alpha) + lambda*(Y'*alpha) +
            0.5*rho*(Y'*alpha)^2;

```

```

25     for ls = 1:20
26         f_new = 0.5*(alpha_try'*(Q*alpha_try)) - sum(alpha_try) + ...
27             lambda*(Y'*alpha_try) + 0.5*rho*(Y'*alpha_try)^2;
28         if f_new <= f_old + 1e-12
29             break;
30         end
31         step = step * 0.5;
32         alpha_try = min(C, max(0, alpha - step*grad));
33     end
34     alpha = alpha_try;
35     % BB步长更新
36     s = alpha - a_old;
37     if norm(s,2) > 0
38         g_new = Q*alpha - ones(m,1) + (lambda + rho*(Y'*alpha))*Y;
39         yk = g_new - g_old;
40         denom = s' * yk;
41         if abs(denom) > 1e-12
42             step = max(1e-6, min(1, (s' * s) / denom));
43         end
44     end
45     % 内层停止
46     if norm(alpha - a_old, inf) < tol
47         break;
48     end
49 end
50 eq = Y' * alpha;
51 lambda = lambda + rho * eq;
52 % 外层自适应增大rho, 强化等式约束
53 if abs(eq) > 10*tol
54     rho = min(rho * 1.5, 1e4);
55 end
56 if abs(eq) < tol
57     break;
58 end
59 end
60 w = X' * (alpha .* Y);
61 sv_margin = find(alpha > 1e-6 & alpha < C-1e-6);
62 if ~isempty(sv_margin)
63     b = mean(Y(sv_margin) - X(sv_margin,:)*w);
64 else
65     sv = find(alpha > 1e-6);
66     if isempty(sv)
67         b = 0;
68     else
69         b = mean(Y(sv) - X(sv,:)*w);
70     end
71 end
72 end

```

ALM 实现

4.2 实验结果

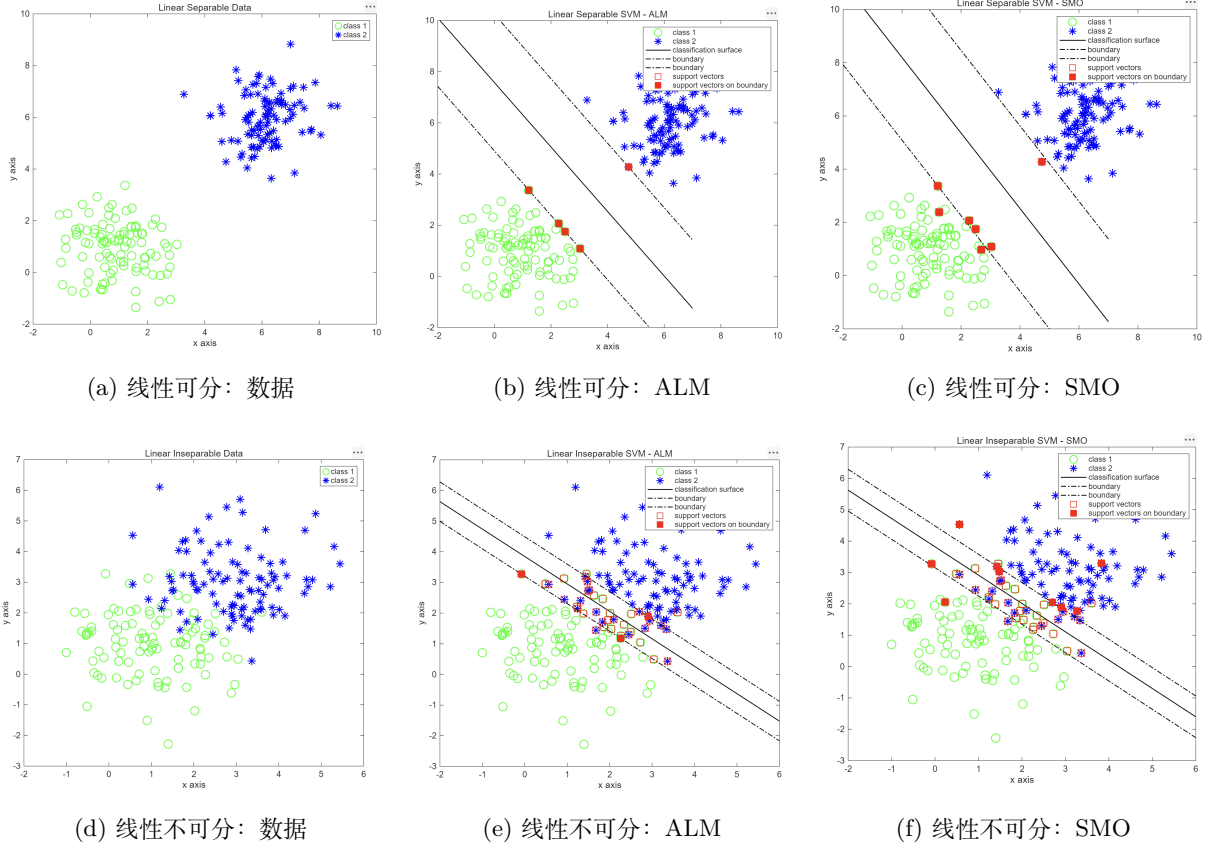


图 1: 线性可分/不可分场景下, ALM 与 SMO 分类结果对比

4.3 基础题结果分析

1. 在线性可分数据上, ALM 与 SMO 都能得到清晰的分类超平面, 训练样本基本可正确分开, 支持向量主要分布在两类样本的边界附近。
2. 在线性不可分数据上, 分类边界会穿过两类样本的重叠区域, 出现少量不可避免的误分, 这是软间隔 SVM 的正常现象。
3. 对比两种方法, SMO 在本实验中收敛更稳定; ALM 对参数 (β 、步长) 更敏感, 参数不合适时边界会有波动。
4. 从几何上看, C 固定时, SVM 在“间隔最大化”与“分类误差惩罚”之间取得折中, 因此不可分场景下边界不会追求训练集完全零误差。

5 附加题

5.1 附加题 1: SVM 的理论推导

5.1.1 间隔最大化如何建成 SVM 模型

1. 函数间隔与几何间隔. 对于二分类样本 (x_i, y_i) (其中 $y_i \in \{+1, -1\}$) 和超平面

$$f(x) = w^\top x + b = 0,$$

样本点的函数间隔定义为

$$\hat{\gamma}_i = y_i(w^\top x_i + b).$$

训练集 T 的函数间隔定义为

$$\hat{\gamma} = \min_{i=1, \dots, N} \hat{\gamma}_i.$$

函数间隔会随 (w, b) 的同比例缩放而变化, 因此不能直接反映点到超平面的绝对距离。为此定义几何间隔

$$\gamma_i = \frac{\hat{\gamma}_i}{\|w\|} = \frac{y_i(w^\top x_i + b)}{\|w\|},$$

以及训练集几何间隔

$$\gamma = \min_{i=1, \dots, N} \gamma_i = \frac{\hat{\gamma}}{\|w\|}.$$

2. 硬间隔 (线性可分) 最大化. SVM 的核心思想是最大化最小几何间隔, 即

$$\max_{w, b} \gamma \quad \text{s.t.} \quad y_i \left(\frac{w^\top}{\|w\|} x_i + \frac{b}{\|w\|} \right) \geq \gamma, \quad \forall i$$

利用尺度不变性, 可令函数间隔规范化为 $\hat{\gamma} = 1$, 得到等价约束

$$y_i(w^\top x_i + b) \geq 1, \quad \forall i.$$

此时最大化几何间隔等价于最小化 $\|w\|$, 进一步写成标准凸二次规划:

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1, \quad \forall i.$$

满足等号

$$y_i(w^\top x_i + b) = 1$$

的样本称为支持向量。对应两条间隔边界超平面为

$$H_1: w^\top x + b = 1, \quad H_2: w^\top x + b = -1,$$

两边界间距 (margin) 为

$$\frac{2}{\|w\|}.$$

3. 软间隔（线性不可分）最大化. 当数据线性不可分时，引入松弛变量 $\xi_i \geq 0$ ，允许样本在一定程度上违反间隔约束：

$$y_i(w^\top x_i + b) \geq 1 - \xi_i, \forall i.$$

并通过惩罚项控制违例代价，得到软间隔 SVM 原始问题：

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \xi_i \geq 0, \forall i. \end{aligned}$$

其中参数 $C > 0$ 用于平衡“间隔最大化”与“误分类惩罚”：

- C 大：更重视减少训练误差，间隔倾向变窄；
- C 小：更重视间隔平滑与泛化，允许更多违例。

5.1.2 对偶问题的求解

1. 硬间隔 SVM 的对偶推导. 硬间隔 SVM 原始问题为

$$\min_{w, b} \quad \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w^\top x_i + b) - 1 \geq 0, \quad i = 1, \dots, N.$$

对每个不等式约束引入拉格朗日乘子 $\alpha_i \geq 0$ ，记

$$\alpha = (\alpha_1, \alpha_2, \dots, \alpha_N)^\top,$$

拉格朗日函数为

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^N \alpha_i (y_i(w^\top x_i + b) - 1).$$

展开得

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^N \alpha_i y_i (w^\top x_i + b) + \sum_{i=1}^N \alpha_i.$$

根据 KKT 驻点条件，对 w, b 求偏导并令其为零：

$$\nabla_w L = 0 \Rightarrow w = \sum_{i=1}^N \alpha_i y_i x_i,$$

$$\nabla_b L = 0 \Rightarrow \sum_{i=1}^N \alpha_i y_i = 0.$$

将其代回 L ，得到对偶函数

$$g(\alpha) = \min_{w, b} L(w, b, \alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i^\top x_j).$$

因此硬间隔 SVM 对偶问题为

$$\max_{\alpha} \quad \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i^\top x_j)$$

$$\text{s.t.} \quad \sum_{i=1}^N \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, \dots, N.$$

求得最优解 α^* 后:

$$w = \sum_{i=1}^N \alpha_i^* y_i x_i.$$

任选一个满足 $\alpha_j^{>0}$ 的支持向量（理想情况下取边界向量）可计算

$$b = y_j - \sum_{i=1}^N \alpha_i^* y_i (x_i^\top x_j).$$

最终分类决策函数为

$$f(x) = \text{sign} \left(\sum_{i=1}^N \alpha_i^* y_i (x^\top x_i) + b \right).$$

2. 软间隔 SVM 的对偶推导. 当数据线性不可分时，引入松弛变量 $\xi_i \geq 0$ ，原始问题为

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i$$

$$\text{s.t.} \quad y_i (w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, N.$$

分别对两组不等式约束引入乘子 $\alpha_i \geq 0$, $\mu_i \geq 0$ ，拉格朗日函数:

$$L(w, b, \xi, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i (y_i (w^\top x_i + b) - 1 + \xi_i) - \sum_{i=1}^N \mu_i \xi_i.$$

对 w, b, ξ_i 求驻点条件:

$$\nabla_w L = 0 \Rightarrow w = \sum_{i=1}^N \alpha_i y_i x_i,$$

$$\nabla_b L = 0 \Rightarrow \sum_{i=1}^N \alpha_i y_i = 0,$$

$$\nabla_{\xi_i} L = 0 \Rightarrow C - \alpha_i - \mu_i = 0 \Rightarrow \alpha_i + \mu_i = C.$$

结合 $\mu_i \geq 0$ 得

$$0 \leq \alpha_i \leq C.$$

代回后得到软间隔对偶问题:

$$\max_{\alpha} \quad \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i^\top x_j)$$

$$\text{s.t.} \quad \sum_{i=1}^N \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, \dots, N.$$

软间隔下关键 KKT 条件为

$$\alpha_i^* (y_i (w^{*\top} x_i + b^*) - 1 + \xi_i^*) = 0,$$

$$\mu_i^* \xi_i^* = 0, \quad \mu_i^* = C - \alpha_i^*.$$

由此可将样本分为:

- $\alpha_i^* = 0$: 样本位于间隔外, 非支持向量;
- $0 < \alpha_i^* < C$: 样本在间隔边界上 (常用于计算 b);
- $\alpha_i^* = C$: 样本在间隔内或被误分。

实践中常用边界支持向量集合 $\mathcal{S} = \{i \mid 0 < \alpha_i^{<C}\}$ 求偏置:

$$b = \frac{1}{|\mathcal{S}|} \sum_{j \in \mathcal{S}} \left(y_j - \sum_{i=1}^N \alpha_i^{y_i(x_i^\top x_j)} \right).$$

对偶求解流程.

1. 构造核 (线性情形为内积) 矩阵 $K_{ij} = x_i^\top x_j$;
2. 求解对偶二次规划, 得到 α^* (满足等式与盒约束);
3. 由 $w^* = \sum_i \alpha_i^* y_i x_i$ 计算法向量 (线性 SVM);
4. 由 $0 < \alpha_i^* < C$ 的边界支持向量估计 b^* ;
5. 得到分类器

$$f(x) = \text{sign} \left(\sum_{i=1}^N \alpha_i^* y_i K(x_i, x) + b^* \right).$$

5.1.3 如何定义支持向量

表 1: 硬间隔与软间隔下支持向量特性对比

维度	线性可分 (Hard Margin)	线性不可分 (Soft Margin)
位置	严格落在间隔边界上	边界上、边界内、甚至错误一侧
判定条件	$y_i(w^\top x_i + b) = 1, \alpha_i > 0$	$\alpha_i > 0$
点数	通常较少	较多 (包含所有违规点)

5.2 附加题 2：学习增广拉格朗日法并调节 β 与步长

5.2.1 增广拉格朗日法 (ALM) 推导笔记

1. 标准约束优化问题. 考虑带等式约束的问题:

$$\min_x f(x) \quad \text{s.t.} \quad h(x) = 0.$$

普通拉格朗日函数为

$$\mathcal{L}(x, \lambda) = f(x) + \lambda^\top h(x).$$

其难点是: 仅靠线性乘子项, 数值迭代时对约束违反的“曲率惩罚”不够, 可能震荡或收敛慢。

2. 增广拉格朗日函数. 在普通拉格朗日函数基础上加入二次罚项:

$$\mathcal{L}_\rho(x, \lambda) = f(x) + \lambda^\top h(x) + \frac{\rho}{2} \|h(x)\|^2, \quad \rho > 0.$$

其中:

- λ : 拉格朗日乘子 (对偶变量);
- ρ : 增广参数 (惩罚强度, 实验中常记作 β 或 rho)。

二次项使约束违反被更强惩罚, 同时仍保留对偶更新机制。

3. ALM 迭代框架. 第 k 轮迭代分两步:

$$x^{k+1} = \arg \min_x \mathcal{L}_\rho(x, \lambda^k),$$

$$\lambda^{k+1} = \lambda^k + \rho h(x^{k+1}).$$

解释: 先在当前乘子下最小化增广拉格朗日, 再用约束残差更新乘子。若 $h(x^{k+1}) \rightarrow 0$, 则乘子趋于稳定。

4. 在 SVM 对偶问题中的形式. 线性 SVM 对偶可写为

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \alpha^\top Q \alpha - \mathbf{1}^\top \alpha \\ \text{s.t.} \quad & y^\top \alpha = 0, \quad 0 \leq \alpha_i \leq C. \end{aligned}$$

把等式约束记为

$$h(\alpha) = y^\top \alpha.$$

则增广拉格朗日为

$$\mathcal{L}_\rho(\alpha, \lambda) = \frac{1}{2} \alpha^\top Q \alpha - \mathbf{1}^\top \alpha + \lambda (y^\top \alpha) + \frac{\rho}{2} (y^\top \alpha)^2.$$

其中盒约束 $0 \leq \alpha_i \leq C$ 通常通过投影处理。

5. 内层梯度与投影更新. 对 α 求梯度:

$$\nabla_{\alpha} \mathcal{L}_\rho = Q\alpha - \mathbf{1} + (\lambda + \rho y^\top \alpha) y.$$

可用投影梯度一步:

$$\alpha \leftarrow \Pi_{[0, C]} \left(\alpha - \eta \nabla_{\alpha} \mathcal{L}_\rho \right),$$

其中 η 为步长, $\Pi_{[0, C]}$ 表示逐元素截断到区间 $[0, C]$ 。

外层再更新乘子:

$$\lambda \leftarrow \lambda + \rho (y^\top \alpha).$$

6. 参数作用与实验现象.

- ρ (或 β) 增大: 更强调满足等式约束 $y^\top \alpha = 0$, 但过大可能导致数值不稳;
- 步长 η 过小: 收敛慢; 过大: 易震荡;
- 适中的 ρ 与步长通常带来更快、稳定的收敛。

5.2.2 代码说明

本实验通过网格扫描 β (程序中为 `rho`) 和步长 `step`, 统计时间、训练精度、等式约束残差:

```
1 function out_table = run_alm_grid(X, Y, params, beta_list, step_list)
2 rows = numel(beta_list) * numel(step_list);
3 beta_col = zeros(rows,1);
4 step_col = zeros(rows,1);
5 time_col = zeros(rows,1);
6 acc_col = zeros(rows,1);
7 res_col = zeros(rows,1);
8 sv_col = zeros(rows,1);
9 k = 0;
10 for i = 1:numel(beta_list)
11     for j = 1:numel(step_list)
12         k = k + 1;
13         p = params;
14         p.rho = beta_list(i);
15         p.step = step_list(j);
16         tic;
17         [w,b,alpha] = train_svm_alm(X, Y, p);
18         t = toc;
19         pred = sign(X*w + b);
20         pred(pred==0) = 1;
21         acc = mean(pred == Y);
22         res = abs(Y' * alpha);
23         sv = sum(alpha > 1e-6);
24         beta_col(k) = p.rho;
25         step_col(k) = p.step;
26         time_col(k) = t;
27         acc_col(k) = acc;
28         res_col(k) = res;
29         sv_col(k) = sv;
30     end
31 end
32 out_table = table(beta_col, step_col, time_col, acc_col, res_col, sv_col, ...
33     'VariableNames',
34     {'beta','step','time_sec','train_acc','eq_residual','sv_count'});
35 end
```

附加题 2 代码 (参数扫描)

5.2.3 实验结果

命令行窗口					
附加题2结果 (ALM: beta/rho 与 step 对速度影响)					
beta	step	time_sec	train_acc	eq_residual	sv_count
1	0.001	0.16106	0.94	1.2602e-05	37
1	0.01	0.11074	0.94	4.7431e-05	37
1	0.05	0.10746	0.94	2.6247e-05	37
5	0.001	0.086641	0.94	1.4648e-05	37
5	0.01	0.12079	0.94	2.9259e-05	37
5	0.05	0.13294	0.94	7.6885e-05	37
20	0.001	0.067761	0.94	9.9212e-05	37
20	0.01	0.065968	0.94	5.0727e-05	37
20	0.05	0.071937	0.94	8.5952e-05	37

图 2: 附加题 2 参数扫描结果

根据参数扫描结果,在 9 组实验中训练精度均为 0.94,支持向量数量均为 37,说明在该数据集与当前参数范围内, β 与步长主要影响的是**优化速度与约束收敛过程**,而不是最终分类性能。从运行时间看,最慢组合约为 ($\beta = 1, step = 0.001$),耗时约 0.16106s;最快组合约为 ($\beta = 20, step = 0.01$),耗时约 0.065968s。可见适当增大 β 可加快约束满足过程,但步长过小会降低收敛速度。等式约束残差 `eq_residual` 均在 10^{-5} 量级,表明各组参数均达到较好的可行性。综合速度与稳定性, $\beta \in [5, 20]$ 、`step` 取中等量级(如 0.01)更合适。

1. 当步长过小(如 `step=0.001`)时,算法更新幅度小,收敛稳定但速度慢。
2. 当步长过大(如 `step=0.05`)时,虽然前期下降快,但容易震荡,导致整体耗时不一定最优。
3. β 过小时,等式约束 $\sum \alpha_i y_i = 0$ 满足较慢; β 过大时,约束项过强,可能引起数值不稳定。
4. 因此存在折中参数区间,可兼顾稳定性与收敛速度。

5.3 附加题 3：调节参数 C ，观察分类效果

5.3.1 代码说明

通过改变 C 的取值，记录训练精度、支持向量数量和 $\|w\|$ ：

```
1 function out_table = run_c_grid(X, Y, method, params, c_list)
2 rows = numel(c_list);
3 C_col = zeros(rows,1);
4 acc_col = zeros(rows,1);
5 sv_col = zeros(rows,1);
6 w_norm_col = zeros(rows,1);
7 for i = 1:numel(c_list)
8     p = params;
9     p.C = c_list(i);
10    [w,b,alpha] = train_svm_choose(X, Y, method, p);
11
12    pred = sign(X*w + b);
13    pred(pred==0) = 1;
14
15    C_col(i) = c_list(i);
16    acc_col(i) = mean(pred == Y);
17    sv_col(i) = sum(alpha > 1e-6);
18    w_norm_col(i) = norm(w);
19 end
20
21 out_table = table(C_col, acc_col, sv_col, w_norm_col, ...
22     'VariableNames', {'C','train_acc','sv_count','w_norm'});
23 end
24
25 function plot_c_effect(X, Y, c_list, method, params)
26 n = size(X,1)/2;
27 figure(5);
28 set(gcf, 'Position', [50,50,1200,900], 'color','w');
29 for i = 1:numel(c_list)
30     p = params;
31     p.C = c_list(i);
32     [w,b,alpha,C] = train_svm_choose(X, Y, method, p);
33     x1 = -2 : 0.02 : 7;
34     y1 = ( -b * ones(1,length(x1)) - w(1) * x1 )/w(2);
35     y2 = ( ones(1,length(x1)) - b * ones(1,length(x1)) - w(1) * x1 )/w(2);
36     y3 = ( -ones(1,length(x1)) - b * ones(1,length(x1)) - w(1) * x1 )/w(2);
37     pred = sign(X*w + b);
38     pred(pred==0) = 1;
39     acc = mean(pred == Y);
40     subplot(2,2,i);
41     plot(X(1:n,1),X(1:n,2),'go','LineWidth',1,'MarkerSize',7); hold on;
42     plot(X(n+1:2*n,1),X(n+1:2*n,2),'b*','LineWidth',1,'MarkerSize',7); hold on;
43     plot(x1,y1,'k','LineWidth',1); hold on;
44     plot(x1,y2,'k-.','LineWidth',1); hold on;
45     plot(x1,y3,'k-.','LineWidth',1); hold on;
```

```

46 plot(X(alpha>0,1),X(alpha>0,2),'rs','LineWidth',1,'MarkerSize',7); hold on;
47 plot(X(alpha<C & alpha>0,1),X(alpha<C & alpha>0,2), ...
48      'rs','MarkerFaceColor','r','LineWidth',1,'MarkerSize',7); hold on;
49 xlabel('x axis'); ylabel('y axis');
50 title(['C=', num2str(c_list(i)), ', acc=', num2str(acc, '%.3f')]);
51 legend('class 1','class 2','surface','+1 margin','-1 margin','sv','margin sv',
52        ...
53        'Location','best');
54 axis tight;
55 end
end

```

附加题 3 代码

5.3.2 实验结果

附加题3结果 (C 对分类效果影响)

C	train_acc	sv_count	w_norm
0.1	0.915	58	1.1253
1	0.925	45	1.4081
10	0.92	44	1.5419
100	0.92	46	1.4702

图 3: 附加题 3 数值结果

由图 3 可见, 当 C 从 0.1 增大到 1 时, 训练准确率由 0.915 提升到 0.925, 支持向量数由 58 降至 45, 说明适度增大惩罚系数可减少欠拟合并使决策边界更聚焦于关键样本。当 C 继续增大到 10 与 100 时, 训练准确率基本维持在 0.92 左右, 支持向量数变化不大 (44 ~ 46), 性能提升趋于饱和, 表明模型已接近该数据集上的可达到上限

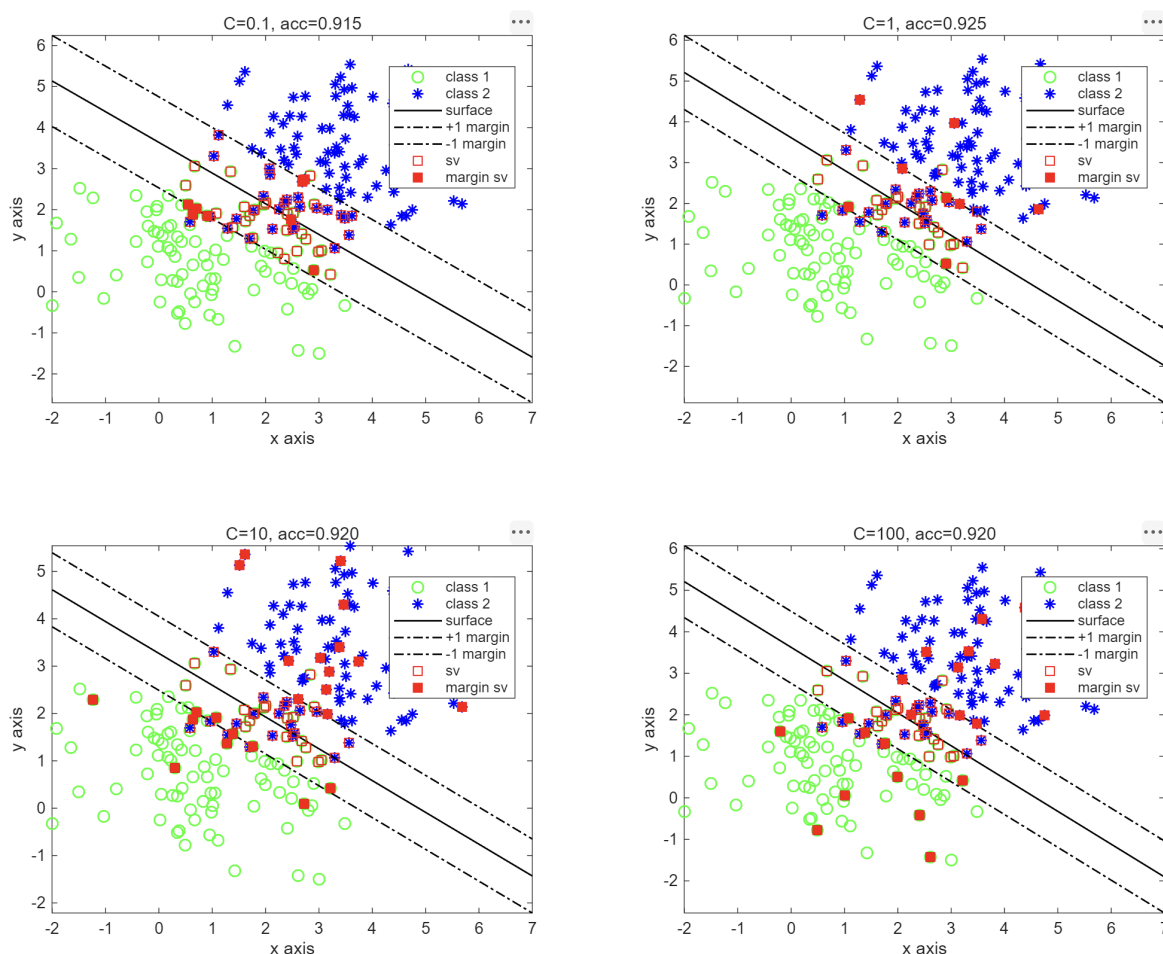


图 4: 附加题 3 分类效果 (不同 C)

图 4 中的四组可视化也显示: 不同 C 下分类线方向整体一致, 但边界位置、间隔宽度和支持向量个数有所不同。总体上, 本实验中 $C = 1$ 的表现最好 (训练精度最高), 说明中等惩罚强度在该任务上取得了较好的间隔-误差折中。

1. C 较小时 (如 $C = 0.1$), 模型对误分类惩罚较弱, 间隔更“软”, 边界更平滑, 但可能欠拟合。
2. C 增大到中等值 (如 $C = 1, 10$) 时, 模型在间隔与误差之间取得较好平衡, 分类效果通常更稳定。
3. C 很大时 (如 $C = 100$), 模型会更强地压低训练误差, 可能导致对噪声更敏感, 泛化风险上升。
4. 支持向量数量和边界位置随 C 变化而改变, 说明 C 直接控制了模型复杂度与容错能力。

6 实验总结收获

本实验完成了线性可分与线性不可分两类数据下的 SVM 建模与可视化。通过 SMO/ALM 双方法实现, 验证了支持向量、间隔边界与分类超平面的关系。附加实验显示: ALM 对 β 与步长较敏感, 存在速度与稳定性权衡; C 直接影响间隔宽度与误分类惩罚强度, 是控制欠拟合/过拟合的重要参数。整体上, SVM 相较感知机在间隔最大化和泛化能力方面表现更优。